■ AmritChidiya — Comprehensive 4-Member Team Role Division & Ownership Handbook

Project: *AmritChidiya (■■■■■■■■■■■) — "Apni Sone Ki Chidiya Ko Phir Se Udaan Do" ■■*

Document Purpose: *Complete technical specification and task division breaking down codebase ownership, step-by-step execution for each role, viva talking points, and standout technical USPs across* ***4 team members*** *(explained so clearly that any beginner can understand).*

---

1. The Big Picture: How the 4 Members Work Together (For Beginners)

Think of AmritChidiya like a modern, highly secure digital government office. Each member is the sole engineer and owner of one critical department:

```
graph TD
    M1["Member 1: The Citizen Counter<br><b>Frontend UI/UX & Citizen Access Lead</b><br>USP: Phygital CSC Dossier Bridge"]
    M2["Member 2: The Ears & Speaking Voice<br><b>Backend API & Speech Processing Lead</b><br>USP: Low-Latency Vernacular Audio Pipeline"]
    M3["Member 3: The Scheme Expert Brain<br><b>AI Agent, Scheme Logic & Cyber Safety Lead</b><br>USP: Anti-Hallucination Matcher & Cyber Shie
    M4["Member 4: The Bank Vault & Cloud Guard<br><b>Database, Auth Security & DevOps Lead</b><br>USP: Zero-Trust Hybrid Cloud & Bcrypt Auth"]

    M1 -->|HTTP POST /chat (JWT Protected)| M2
    M1 -->|Audio Upload /transcribe & /tts| M2
    M2 -->|Invoke State Machine| M3
    M3 -->|Resolve Scheme URLs & Cyber Rules| M2
    M2 -->|Supabase / SQLite Failover CRUD| M4
    M1 -->|Client JWT & Session Sync| M4
```

- **Member 1 (The Front Counter):** Builds the welcoming touch-screen interface where the citizen talks, checks their eligibility, and receives a printed receipt to take home.
- **Member 2 (The Ears & Voice):** Listens to the citizen's voice in regional Indian accents, converts it into text at lightning speed, and speaks back with a human-sounding voice.
- **Member 3 (The Scheme Expert & Security Detective):** Knows every single government rule, guarantees 0% false promises, and checks suspicious WhatsApp links to protect citizens from scams.
- **Member 4 (The Bank Vault & Cloud Guard):** Locks up user passwords with unhackable 12-round Bcrypt encryption, gives out secure digital entry passes (JWT tokens), and keeps a backup database running 24/7.

---

2. Technical Responsibility & Deliverables Matrix

Member	Department & Role	Core Assigned Files	Member Standout Technical USP

Member 1	**Frontend UI/UX & Citizen Access Lead**	`frontend/app/page.tsx`
		`frontend/components/CscDossierModal.tsx`
		`frontend/components/EligibilityCalculatorModal.tsx`
		`frontend/components/MarkdownContent.tsx`
		`frontend/app/globals.css`
			Phygital Last-Mile Bridge & Citizen CSC Dossier Generator
			Bridges the digital-to-physical divide for rural India by generating structured, printable **CSC Scheme Dossiers** (`AC-CSC-XXXXXX`) with NPCI/DBT bank checklists & VLE operator sign-off boxes for village Jan Seva Kendras, coupled with a responsive Next.js 16 UI and hands-free Talk Mode.

Member 2	**Backend API & Speech Processing Lead**	`backend/main.py`
		`backend/requirements.txt`
		Groq Whisper API & Edge-TTS
			Low-Latency Vernacular Audio Pipeline with Indian Accent DSP
			Engineered an end-to-end voice pipeline using Groq Whisper Large v3 with phonetic domain bias for Indian accents, ambient silence/hallucination suppression, and high-fidelity Microsoft Edge-TTS neural streaming in 5 Indian languages.

Member 3	**AI Agent, Scheme Logic & Cyber Safety Lead**	`backend/agents/chat_agent.py`
		`backend/prompts/system_prompt.py`
		`frontend/components/ScamShieldModal.tsx`
			Anti-Hallucination Welfare Match Engine & Domain Cyber Shield
			Built the LangGraph dual-phase state machine that prevents welfare program hallucinations through strict intake rules, paired with the **Official Domain Shield & Scam Checker** that cryptographically verifies `.gov.in`/`.nic.in` domains to protect citizens from WhatsApp phishing.

Member 4	**Database, Auth Security & DevOps Lead**	`backend/database.py`
		`frontend/lib/auth.ts`
		`backend/supabase_schema.sql`
			Zero-Trust Hybrid Cloud Persistence with Bcrypt Salted Auth
			Architected an enterprise-grade dual-database architecture with Supabase Cloud PostgreSQL and instant local SQLite failover, hardened by salted **Bcrypt (12 rounds)**, signed **24-hour JWT tokens**, SlowAPI rate limiting, and production CORS deployment.

3. Member 1: Frontend UI/UX & Citizen Access Lead

■ Member USP (Unique Selling Proposition)

"Phygital Last-Mile Bridge & Citizen CSC Dossier Generator"

Why this matters for a beginner: *Most welfare applications fail in rural India because citizens don't own laptops or printers to apply online. Member 1 bridged this gap by inventing the* ***Phygital CSC Dossier Engine****: an in-app tool that generates an official printable receipt (`AC-CSC-XXXXXX`) with their verified schemes, an NPCI/DBT bank checklist, and a physical CSC VLE Operator verification stamp box. A rural citizen can simply print this sheet or take it on their phone to their local Jan Seva Kendra / Cyber Cafe to complete their application with zero confusion.*

■ Assigned Codebase Files

- **frontend/app/page.tsx** — Main page component, UI layout, state machine, event handlers.
- **frontend/components/CscDossierModal.tsx** — Printable Common Service Center (CSC) scheme dossier receipt with custom print stylesheets (`window.print()`).
- **frontend/components/EligibilityCalculatorModal.tsx** — Real-time multi-criteria scheme eligibility calculator evaluating State, Category, Income, and Profession with direct official portal links.
- **frontend/components/MarkdownContent.tsx** — Custom markdown parser rendering secure external links (`target="_blank" rel="noopener noreferrer"`).
- **frontend/app/globals.css** — Custom scrollbars, glassmorphism design tokens, gold pulsing orb animations.

■■ Step-by-Step Working Mechanism

1. **Language Selection:** Citizen clicks Hindi or Tamil. `page.tsx` switches the greeting, UI labels, and speech voice model instantly without page reload.
2. **Instant Yojana Calculator:** In `EligibilityCalculatorModal.tsx`, the citizen picks simple dropdowns (State, Category, Income, Profession). The code dynamically calculates matching welfare schemes and shows estimated annual cash benefits (e.g. ■30,000–■55,000/yr).
3. **Seamless Context Transfer:** Clicking *"Get Step-by-Step Guide →"* passes their profile into the chat state machine without retyping.
4. **Phygital CSC Printing:** In `CscDossierModal.tsx`, custom `@media print` CSS formats an A4 dossier hiding browser bars and showing official VLE sign-off boxes.
5. **Safe Hyperlinks:** `MarkdownContent.tsx` ensures all official government links open safely in a new tab with `rel="noopener noreferrer"`.

■ Interview / Viva Questions & Model Answers

- **Q1: What does 'Phygital' mean and why is the CSC Dossier unique?**

Answer: 'Phygital' combines Physical + Digital. Most rural citizens must visit a physical Common Service Center (CSC) to complete biometric/Aadhaar verification. The CSC Dossier organizes their documents, scheme codes, and DBT checklist in advance, preventing application rejections.

- **Q2: How do you handle print styling in Next.js?**

Answer: Using CSS `@media print` rules inside `CscDossierModal.tsx`. It hides the sidebar, input bar, and buttons, converting dark-mode glassmorphism into high-contrast black-and-white print ready for standard A4 printers.

---

4. Member 2: Backend API & Speech Processing Lead

■ Member USP (Unique Selling Proposition)

"Low-Latency Vernacular Audio Pipeline with Indian Accent DSP & Neural Speech"

Why this matters for a beginner: *Standard speech recognition fails on Indian accents, mixed dialects (Hinglish), and noisy environments. Member 2 created a dedicated audio pipeline using Groq Whisper Large v3 primed with custom Indian regional vocabulary hints (UP, Uttarakhand, B.Tech, OBC, scholarship, 1.5 lakh), ambient noise/silence hallucination suppression, and Microsoft Edge-TTS streaming natural human voice in 5 Indian languages.*

■ Assigned Codebase Files

- **backend/main.py** — FastAPI web service, STT audio transcoding pipeline, Edge-TTS streaming handler, CORS origin regex middleware.
- **backend/requirements.txt** — Dependency specification### ■■ Step-by-Step Working Mechanism

1. **Voice Capture & Ephemeral Upload:** Citizen speaks $\rightarrow$ browser records a WebM audio blob $\rightarrow$ POSTed to `/transcribe` in `backend/main.py`. The file is saved inside a `tempfile.NamedTemporaryFile` and is strictly deleted in a `finally: os.unlink()` block, ensuring citizen voice recordings never linger on disk.
2. **Regional Phonetic Prompting:** Member 2 supplies Whisper with an exact prompt of Indian states, categories, and education levels so Whisper doesn't mistranslate Indian terms into English.
3. **Silence Hallucination Cleansing:** Whisper often hallucinates phrases like *"Thank you for watching"* when someone pauses. Member 2's code checks an exact blacklist of silence artifacts and strips them cleanly.
4. **Text-to-Speech Sanitization:** The AI response has markdown asterisks (**`bold`**), hashtags (`###`), and links. Member 2's `clean_text_for_speech` function removes all markup so Edge-TTS reads smooth, lifelike sentences.
5. **FastAPI Routing & Rate Limiting:** Hosts `/chat`, `/transcribe`, `/tts`, `/signup`, `/login`, enforces CORS origin regex (`^https://.*\.vercel\.app$`), and blocks DDoS and credential attacks with SlowAPI rate limits (20/min for STT, 30/min for TTS).

■ Interview / Viva Questions & Model Answers

- **Q1: Why did you choose Groq Whisper over other Speech-to-Text APIs?**

Answer: Groq runs on specialized LPU (Language Processing Unit) chips, providing sub-500ms transcription latency. This makes voice conversations feel real-time rather than having an awkward 3-second delay.

- **Q2: How do you protect user privacy for voice recordings?**

Answer: Audio files are treated as ephemeral streams. They are created in temporary OS memory/disk via `tempfile.NamedTemporaryFile` and unconditionally unlinked (`os.unlink()`) inside a `finally` block immediately after transcription finishes.

- **Q3: How does the text sanitization function work before TTS?**

Answer: `clean_text_for_speech()` uses regular expressions to strip out markdown asterisks, hashes, code blocks, and URLs. It turns bullet points into natural sentence pauses so the voice sounds like a caring human companion.

---

5. Member 3: AI Agent, Scheme Logic & Cyber Safety Lead

■ Member USP (Unique Selling Proposition)

"Anti-Hallucination Welfare Match Engine, Domain Cyber Shield & National Helpline Bridge"

Why this matters for a beginner: *If an AI makes up a fake scholarship or sends a student to an expired link, citizens lose faith in the system. Member 3 built a strict LangGraph state machine that prohibits fake schemes, combined with the* ***Official Domain Shield & Scam Checker*** *(`ScamShieldModal.tsx`) that verifies authentic `.gov.in` domains, teaches the 4 Golden Rules of fraud defense, links directly to the National Cyber Crime Helpline (1930), and protects citizens from fake WhatsApp schemes demanding registration fees.*

■ Assigned Codebase Files

- **backend/agents/chat_agent.py** — LangGraph state graph, scheme extraction regex engine, official government URL resolver.
- **backend/prompts/system_prompt.py** — Core system prompt, role confinement, phase transition protocols, hyperlinked scheme markdown templates.
- **frontend/components/ScamShieldModal.tsx** — Official domain verification tool, 4-rule scam detector, and 1930 Cyber Helpline bridge.

■■ Step-by-Step Working Mechanism

1. **Phase 1 (Profile Intake):** If a student says *"Mujhe scholarship chahiye"*, the agent checks if State, Income, Category, and Education are provided. If missing, it asks polite clarifying questions and suppresses scheme extraction.
2. **Phase 2 (Eligibility Delivery):** Once details are verified, the agent outputs exact scheme names formatted in structured markdown headers.
3. **Government URL Mapper & Fallback:** `SCHEME_URL_MAP` in `chat_agent.py` maps each scheme name directly to official government portals (`scholarships.gov.in`, `pmkisan.gov.in`, `scholarship.up.gov.in`). If an unmapped scheme is detected, it securely defaults to `https://www.myscheme.gov.in/search?q=...` (India's official National Scheme Portal), guaranteeing zero phishing links.
4. **Phase 3 (Step-by-Step Roadmap):** Delivers a 5-step registration breakdown with required document checklists.
5. **Cyber Shield & Fraud Education:** When a citizen pastes a WhatsApp link into `ScamShieldModal.tsx`, the code extracts the hostname, checks for official `.gov.in`/`.nic.in` accreditation, flags phishing TLDs (`.xyz`, `.top`), and displays the 4 Golden Rules with a direct dialer to 1930.

■ Interview / Viva Questions & Model Answers

- **Q1: How does LangGraph prevent LLM hallucinations in government schemes?**

Answer: LangGraph enforces a structured state machine. The prompt explicitly prohibits suggesting schemes during profile intake, and the regex extractor only accepts schemes formatted in strict headers that match verified criteria.

- **Q2: How does the Cyber Shield protect against WhatsApp fraud?**

Answer: It inspects the domain structure. All genuine Indian central and state welfare portals must end in `.gov.in` or `.nic.in`. If a link ends in `.xyz` or asks for private UPI fees, it triggers a high-risk scam alert and points the citizen to the National Cyber Crime Helpline (1930).

6. Member 4: Database, Auth Security & DevOps Lead

■ Member USP (Unique Selling Proposition)

"Zero-Trust Hybrid Cloud Persistence with 12-Round Bcrypt Auth, IDOR Defense & Fail-Safe SQLite Replicas"

Why this matters for a beginner: *User passwords must be protected with bank-grade security, citizen chat histories must never be snooped on by other accounts, and a welfare app must never crash if the internet or cloud goes down. Member 4 built a dual-database architecture (Supabase Cloud PostgreSQL + auto SQLite failover), hardened by salted* ***Bcrypt (12 rounds)*** *with automatic legacy hash migration, signed* ***24-hour HS256 JWT tokens****, strict* ***IDOR ownership checks****,* ***SecurityHeadersMiddleware*** *(nosniff, DENY, XSS), and SlowAPI rate limiting.*

■ Assigned Codebase Files

- **backend/database.py** — Supabase Client API manager, automatic SQLite fallback engine, 12-round bcrypt password hashing with legacy migration, chat history CRUD.
- **backend/supabase_schema.sql** — Supabase PostgreSQL schema with primary keys, foreign keys, cascade deletes, and RLS policies.
- **frontend/lib/auth.ts** — Client authentication state machine, JWT bearer header injector, localStorage cache synchronization.

■■ Step-by-Step Working Mechanism

1. **Bcrypt Password Salting & Migration:** When a user signs up, Member 4's `_hash_password()` generates a unique 12-round cryptographic salt (`bcrypt.gensalt(12)`). For legacy accounts created under older SHA-256 schemas, `_verify_password()` and `_needs_rehash()` detect legacy hashes and upgrade them transparently to bcrypt upon successful login.
2. **Digital Wristbands (JWT Tokens) & IDOR Defense:** Upon login, the backend issues an HS256 signed JWT token valid for 24 hours. The frontend attaches this token in the `Authorization: Bearer <token>` header. Endpoints `/chats/{user_id}` enforce strict ownership (`if current_user["id"] != user_id: raise 403 Forbidden`), completely preventing Insecure Direct Object References.
3. **HTTP Security Hardening:** `SecurityHeadersMiddleware` injects 5 defensive HTTP headers: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `X-XSS-Protection: 1; mode=block`, strict Referrer-Policy, and restrictive Permissions-Policy.
4. **Dual-Database Failover:** `database.py` tries Supabase Cloud PostgreSQL first. If offline or credentials are missing, it routes queries to local SQLite (`amrit_chidiya.db`) without throwing an unhandled exception. Local SQLite files are strictly gitignored to prevent credential leaks.
5. **Rate Limiting (SlowAPI) & DevOps Deployment:** Limits requests (5/min for signup, 10/min for login, 30/min for chat) to prevent bots from crashing the server. Manages production environment variables on Render and Vercel.

■ Interview / Viva Questions & Model Answers

- **Q1: Why is Bcrypt safer than plain SHA-256 for passwords?**

Answer: SHA-256 is designed to be fast, which makes it easy for attackers with GPUs to try billions of guesses per second. Bcrypt is intentionally slow, adaptive, and salted (12 rounds = 4,096 iterations), making brute-force and rainbow table attacks computationally unfeasible.

- **Q2: What is IDOR and how does Member 4 prevent it?**

Answer: IDOR (Insecure Direct Object Reference) happens when a malicious user changes an ID in the URL (e.g. `/chats/user_123` to `/chats/user_456`) to spy on another citizen's private chats. Member 4 prevents this by verifying that the user ID embedded inside the signed JWT token strictly matches the target resource ID, returning `403 Forbidden` if they differ.

- **Q3: Why use a hybrid Supabase + SQLite database?**

Answer: Supabase Cloud allows users to access their saved chats from any phone or computer. But if an internet drop occurs in a rural village, the automatic SQLite fallback guarantees the app continues working locally.